Математическое моделирование

Лабораторная работа № 2

Авдадаев Джамал

Содержание

1 Цель работы **6**

2 Задание **7**

3 Выполнение лабораторной работы **8**
3.1 Теоретические сведения 8
3.2 Задача 14

4 Две системы ОДУ: численное решение и визуализация **15**
4.1 Инициализация проекта и загрузка пакетов 15
4.2 Начальные условия и временной интервал 16
4.3 Параметры первой системы 16
4.4 Параметры второй системы 16
4.5 Внешние воздействия для первой системы 17
4.6 Внешние воздействия для второй системы 17
4.7 Определение моделей 17
4.8 Утилита: один прогон (решение, график, таблица, сохранение) . . 18
4.9 Запуск 1: первая система 19
4.10 Запуск 2: вторая система 19

5 Параметрическое исследование двух систем ОДУ **21**
5.1 Активация проекта и загрузка пакетов 21
5.2 Внешние воздействия для первой системы 22
5.3 Внешние воздействия для второй системы 22
5.4 Определение моделей 22
5.5 Определение базовых параметров в Dict 23
5.6 Функция-обертка для запуска одного эксперимента 24
5.7 Запуск базового эксперимента (линейная система) 26
5.8 Визуализация базового эксперимента 27
5.9 Второй базовый эксперимент (нелинейная система) 27
5.10 Параметрическое сканирование 29
5.11 Запуск всех экспериментов и сбор результатов 30
5.12 Анализ и визуализация результатов сканирования 32
5.13 Бенчмаркинг 36
5.14 Сохранение всех результатов 38
5.15 Базовые эксперименты 40

5.16 Параметрическое сканирование 42
5.17 Время вычислений 44
5.18 Анализ итоговой метрики norm_final 45

6 Выводы **46**

Список литературы **47**

Список иллюстраций

3.1 Жесткая модель войны 11
3.2 Фазовые траектории для второго случая 13

5.1 Базовый эксперимент (linear) 40
5.2 Базовый эксперимент (nonlinear) 41
5.3 Сканирование траекторий x(t) 42
5.4 Сканирование траекторий y(t) 43
5.5 Зависимость времени вычисления 44
5.6 Зависимость norm_final 45

Список таблиц

1 Цель работы

Рассматриваются основные математические модели вооружённых конфликтов, известные как модели Ланчестера. В таких моделях участвуют две противоборствующие стороны, которые могут представлять как регулярные армии, так и нерегулярные формирования. Ключевой характеристикой каждой стороны является её численность.

Если в некоторый момент времени численность одной из сторон становится равной нулю, то считается, что эта сторона проиграла, при условии, что численность другой стороны остаётся положительной..

2 Задание

1. Рассмотреть три варианта моделей Ланчестера.
2. Построить графики изменения численности войск.
3. Определить сторону, одержавшую победу.

3 Выполнение лабораторной работы

3.1 Теоретические сведения

Рассмотрим три возможных сценария вооружённого противостояния:

1. столкновение регулярных армий;
2. конфликт между регулярной армией и партизанскими формированиями;
3. противостояние между партизанскими отрядами.

3.1.1 1. Бой между регулярными армиями

Численность регулярных войск изменяется под влиянием нескольких факторов:

1. естественные потери, не связанные непосредственно с боевыми действиями (болезни, дезертирство, травмы);
2. потери, возникающие в результате вооружённого столкновения сторон;
3. пополнение войск за счёт резервов.

В этом случае динамика численности войск описывается системой дифференциальных уравнений

$$\begin{cases} \frac{dx}{dt} = -a(t)x(t) - b(t)y(t) + P(t) \\ \frac{dy}{dt} = -c(t)x(t) - h(t)y(t) + Q(t) \end{cases}$$

Слагаемые $-a(t)x(t)$ и $-h(t)y(t)$ характеризуют естественные потери. Члены $-b(t)y(t)$ и $-c(t)x(t)$ отражают потери, вызванные непосредственным боевым воздействием противника.

Коэффициенты $b(t)$ и $c(t)$ описывают эффективность боевых действий соответствующих сторон, а параметры $a(t)$ и $h(t)$ учитывают влияние различных небойовых факторов. Функции $P(t)$ и $Q(t)$ моделируют поступление подкреплений.

3.1.2 2. Регулярная армия против партизанских отрядов

Партизанские формирования действуют более скрытно и распределённо по территории. Поэтому воздействие регулярной армии носит менее точечный характер. Предполагается, что темп потерь партизан зависит как от численности армии противника, так и от численности самих партизан.

Модель принимает следующий вид:

$$\begin{cases} \frac{dx}{dt} = -a(t)x(t) - b(t)y(t) + P(t) \\ \frac{dy}{dt} = -c(t)x(t)y(t) - h(t)y(t) + Q(t) \end{cases}$$

3.1.3 3. Конфликт между партизанскими отрядами

Если обе стороны действуют как нерегулярные формирования, то интенсивность потерь определяется произведением численностей противников. Тогда система уравнений имеет форму

$$\begin{cases} \frac{dx}{dt} = -a(t)x(t) - b(t)x(t)y(t) + P(t) \\ \frac{dy}{dt} = -h(t)y(t) - c(t)x(t)y(t) + Q(t) \end{cases}$$

3.1.4 Упрощённая модель Ланчестера

В простейшем случае предполагается, что коэффициенты боевой эффективности постоянны, а подкрепления и небойовые потери отсутствуют. Тогда система принимает вид

$$\begin{cases} \frac{dx}{dt} = -by \\ \frac{dy}{dt} = -ax \end{cases}$$

Здесь предполагается, что каждый солдат армии x выводит из строя c солдат армии y за единицу времени, а каждый солдат армии $y - b$ солдат армии x.

Делением уравнений получаем

$$\frac{dx}{dy} = \frac{by}{cx}$$

После преобразований получаем интеграл движения

$$cx^2 - by^2 = C$$

Следовательно, изменение численностей происходит вдоль гипербол, определяемых данным уравнением.

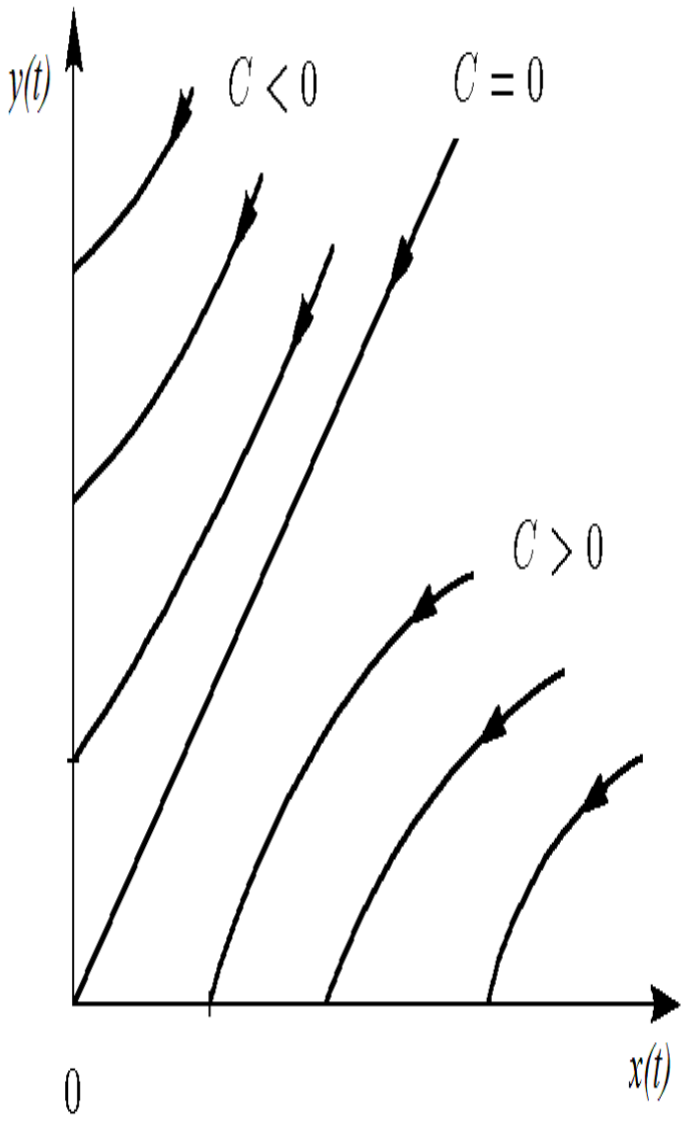

Рисунок 3.1: Жесткая модель войны

Эти гиперболы разделяются прямой

$$\sqrt{cx} = \sqrt{by}$$

Если начальная точка расположена выше этой прямой, то в ходе конфликта численность армии x стремится к нулю и побеждает армия y. Если же начальная точка находится ниже линии разделения, победителем становится армия

x.

В случае расположения начального состояния на самой разделяющей прямой конфликт приводит к взаимному истощению сторон.

Из анализа модели следует важное соотношение: чтобы противостоять противнику, превосходящему по численности в k раз, необходимо иметь вооружение примерно в k^2 раз более эффективное.

Несмотря на сильную идеализацию, модель полезна для качественного анализа динамики конфликта.

3.1.5 Регулярная армия против партизан

При аналогичных упрощениях система принимает вид

$$\begin{cases} \frac{dx}{dt} = -by(t) \\ \frac{dy}{dt} = -cx(t)y(t) \end{cases}$$

Для неё существует интеграл движения

$$\frac{b}{2}x^2(t) - cy(t) = C_1$$

где

$$C_1 = \frac{b}{2}x^2(0) - cy(0)$$

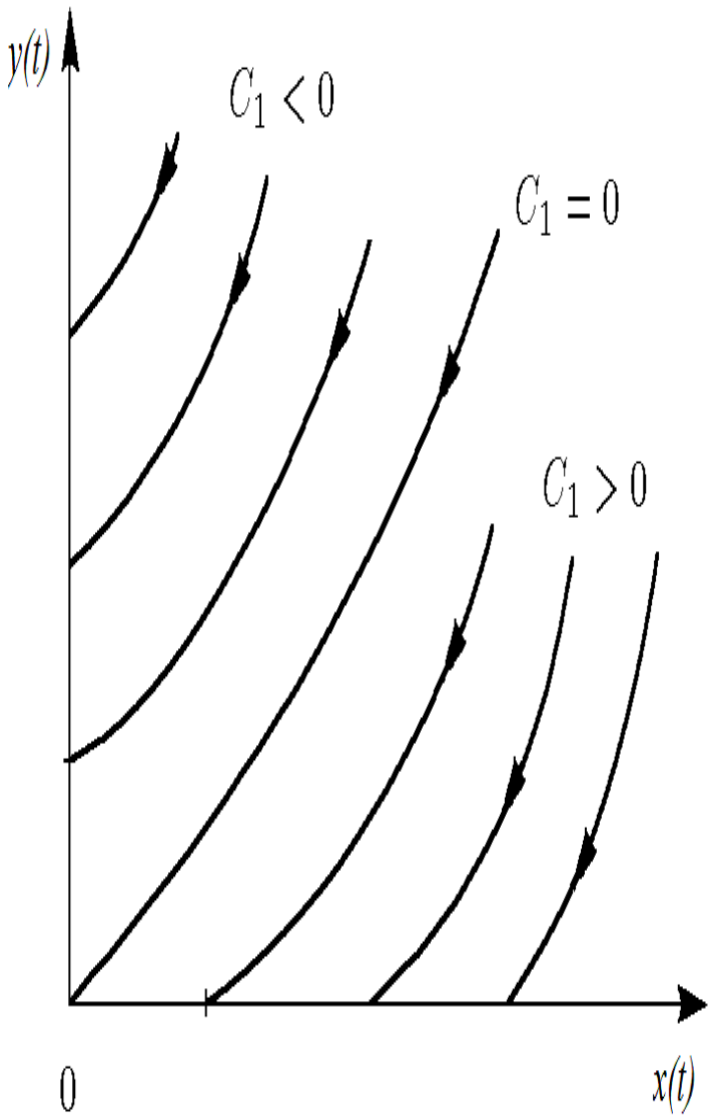

Рисунок 3.2: Фазовые траектории для второго случая

Из анализа фазовых траекторий следует:

- при $C_1 > 0$ победу одерживает регулярная армия;
- при $C_1 < 0$ успех достигают партизанские формирования.

Условие победы регулярной армии можно записать в виде

$$\frac{b}{2}x^2(0) > cy(0)$$

Таким образом, для достижения преимущества партизанам требуется значительно увеличивать как свою численность, так и коэффициент эффективности действий.

3.2 Задача

Рассматривается конфликт между странами X и Y. Численности армий являются функциями времени $x(t)$ и $y(t)$.

Начальные условия:

- армия страны X: 32888 человек;
- армия страны Y: 17777 человек.

Предполагается, что коэффициенты a, b, c, h являются постоянными, а функции $P(t)$ и $Q(t)$ непрерывны.

Необходимо построить графики изменения численностей армий для следующих случаев.

3.2.1 1. Модель регулярных войск

$$\begin{cases} \frac{dx}{dt} = -0.55x(t) - 0.77y(t) + 1.5\sin(3t+1) \\ \frac{dy}{dt} = -0.66x(t) - 0.44y(t) + 1.2\cos(t+1) \end{cases}$$

3.2.2 2. Модель регулярной армии и партизан

$$\begin{cases} \frac{dx}{dt} = -0.27x(t) - 0.88y(t) + \sin(20t) \\ \frac{dy}{dt} = -0.68x(t)y(t) - 0.37y(t) + \cos(10t) \end{cases}$$

Для численного моделирования и построения графиков использовались внешние файлы с программным кодом:

4 Две системы ОДУ: численное решение и визуализация

Цель: решить две системы ОДУ, построить графики решений, сохранить изображения и таблицы с результатами.

4.1 Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE)
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

4.2 Начальные условия и временной интервал

```
x0 = 32888.0
y0 = 17777.0
t0 = 0.0
tmax = 1.0

u0 = [x0, y0]
tspan = (t0, tmax)
```

4.3 Параметры первой системы

```
a = 0.55
b = 0.77
c = 0.66
d = 0.44

p1 = (a = a, b = b, c = c, d = d)
```

4.4 Параметры второй системы

```
a2 = 0.27
b2 = 0.88
c2 = 0.68
d2 = 0.37

p2 = (a = a2, b = b2, c = c2, d = d2)
```

4.5 Внешние воздействия для первой системы

```
function P(t)
    return 1.5 * sin(3 * t + 1)
end

function Q(t)
    return 1.2 * cos(t + 1)
end
```

4.6 Внешние воздействия для второй системы

```
function P2(t)
    return sin(20 * t)
end

function Q2(t)
    return cos(10 * t) + 1
end
```

4.7 Определение моделей

Первая система: dx/dt = -ax - by + P(t) dy/dt = -cx - dy + Q(t)

```
function syst1!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P(t)
    dy[2] = -p.c * y[1] - p.d * y[2] + Q(t)
end
```

Вторая система: dx/dt = -a*x* - *b*y + P2(t) dy/dt = -c*x*y - d*y + Q2(t)

```
function syst2!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P2(t)
    dy[2] = -p.c * y[1] * y[2] - p.d * y[2] + Q2(t)
end
```

4.8 Утилита: один прогон (решение, график, таблица, сохранение)

```
function run_case(case_name; model!, u0, tspan, p, nt=100)
```

— Сетка по времени —

```
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))
```

— Решение ОДУ —

```
    prob = ODEProblem(model!, u0, tspan, p)
    sol = solve(prob, Tsit5(), saveat=tgrid)
```

— Таблица с результатами —

```
    df = DataFrame(
        t = sol.t,
        x = first.(sol.u),
        y = last.(sol.u)
    )
```

— Визуализация —

```
    plt = plot(sol,
        xlabel = "t",
        ylabel = "Значение",
        label = ["x(t)" "y(t)"],
        lw = 2,
        title = "Решение системы — $case_name",
        legend = :topright
    )
```

— Сохранение —

```
    savefig(plt, plotsdir(script_name, "$case_name.png"))
    @save datadir(script_name, "data_$case_name.jld2") df p u0 tspan nt

    return (plt = plt, df = df, sol = sol)
end
```

4.9 Запуск 1: первая система

```
res1 = run_case("system1"; model! = syst1!, u0 = u0, tspan = tspan, p = p1, nt = 100)

println("Система 1 — первые 5 строк:")
println(first(res1.df, 5))
```

4.10 Запуск 2: вторая система

```
res2 = run_case("system2"; model! = syst2!, u0 = u0, tspan = tspan, p = p2, nt = 100)


println("\nСистема 2 — первые 5 строк:")
println(first(res2.df, 5))
```

(опционально) показать последний график в интерактивной среде

```
res2.plt
```

5 Параметрическое исследование двух систем ОДУ

5.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
```

Установка каталогов

```
script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE)
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

5.2 Внешние воздействия для первой системы

```
function P(t)
    return 1.5 * sin(3 * t + 1)
end

function Q(t)
    return 1.2 * cos(t + 1)
end
```

5.3 Внешние воздействия для второй системы

```
function P2(t)
    return sin(20 * t)
end

function Q2(t)
    return cos(10 * t) + 1
end
```

5.4 Определение моделей

Первая система: dx/dt = -ax - by + P(t) dy/dt = -cx - dy + Q(t)

```
function syst_linear!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P(t)
    dy[2] = -p.c * y[1] - p.d * y[2] + Q(t)
end
```

Вторая система: dx/dt = -a*x* - *b*y + P2(t) dy/dt = -c*x*y - d*y + Q2(t)

```
function syst_nonlinear!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P2(t)
    dy[2] = -p.c * y[1] * y[2] - p.d * y[2] + Q2(t)
end
```

5.5 Определение базовых параметров в Dict

model_type управляет выбором системы: :linear -> первая система :nonlinear -> вторая система

```
base_params = Dict(
    :x0 => 32888.0,
    :y0 => 17777.0,

    :tspan => (0.0, 1.0),
    :nt => 100,

    :model_type => :linear,         # :linear или :nonlinear
```

Параметры первой системы

```
    :a => 0.55,
    :b => 0.77,
    :c => 0.66,
    :d => 0.44,
```

Параметры второй системы

```
    :a2 => 0.27,
    :b2 => 0.88,
    :c2 => 0.68,
    :d2 => 0.37,

    :solver => Tsit5(),
    :experiment_name => "base_experiment"
)

println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println(" $key = $value")
end
```

5.6 Функция-обертка для запуска одного эксперимента

```
function run_single_experiment(params::Dict)
    @unpack x0, y0, tspan, nt, model_type, a, b, c, d, a2, b2, c2, d2, solver = param

    u0 = [x0, y0]
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))

    if model_type == :linear
        p = (a=a, b=b, c=c, d=d)
        prob = ODEProblem(syst_linear!, u0, tspan, p)
    else
```

```
        p = (a=a2, b=b2, c=c2, d=d2)
        prob = ODEProblem(syst_nonlinear!, u0, tspan, p)
    end

    sol = solve(prob, solver; saveat=tgrid)

    x_vals = first.(sol.u)
    y_vals = last.(sol.u)
```

— Мини-анализ —

```
    x_final = x_vals[end]
    y_final = y_vals[end]

    x_max_abs = maximum(abs.(x_vals))
    y_max_abs = maximum(abs.(y_vals))

    norm_final = sqrt(x_final^2 + y_final^2)

    return Dict(
        "solution" => sol,
        "t_points" => sol.t,
        "x_values" => x_vals,
        "y_values" => y_vals,

        "x_final" => x_final,
        "y_final" => y_final,
        "x_max_abs" => x_max_abs,
        "y_max_abs" => y_max_abs,
```

```
        "norm_final" => norm_final,

        "parameters" => params
    )
end
```

5.7 Запуск базового эксперимента (линейная система)

```
data_base, path_base = produce_or_load(
    datadir(script_name, "single"),
    base_params,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты базового эксперимента:")
println(" x_final: ", data_base["x_final"])
println(" y_final: ", data_base["y_final"])
println(" x_max_abs: ", data_base["x_max_abs"])
println(" y_max_abs: ", data_base["y_max_abs"])
println(" norm_final: ", round(data_base["norm_final"]; digits=4))
println(" Файл результатов: ", path_base)
```

5.8 Визуализация базового эксперимента

```
p1 = plot(
    data_base["t_points"], data_base["x_values"],
    label="x(t)",
    xlabel="t",
    ylabel="Значение",
    title="Базовый эксперимент (model_type=$(base_params[:model_type]))",
    lw=2,
    legend=:topright,
    grid=true
)
plot!(
    p1,
    data_base["t_points"], data_base["y_values"],
    label="y(t)",
    lw=2
)

savefig(p1, plotsdir(script_name, "single_experiment_linear.png"))
```

5.9 Второй базовый эксперимент (нелинейная система)

```
base_params2 = copy(base_params)
base_params2[:model_type] = :nonlinear
base_params2[:experiment_name] = "base_experiment_nonlinear"
```

```
data_base2, path_base2 = produce_or_load(
    datadir(script_name, "single"),
    base_params2,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты второго базового эксперимента:")
println(" x_final: ", data_base2["x_final"])
println(" y_final: ", data_base2["y_final"])
println(" x_max_abs: ", data_base2["x_max_abs"])
println(" y_max_abs: ", data_base2["y_max_abs"])
println(" norm_final: ", round(data_base2["norm_final"]; digits=4))
println(" Файл результатов: ", path_base2)

p1b = plot(
    data_base2["t_points"], data_base2["x_values"],
    label="x(t)",
    xlabel="t",
    ylabel="Значение",
    title="Базовый эксперимент (model_type=$(base_params2[:model_type]))",
    lw=2,
    legend=:topright,
    grid=true
)
plot!(
```

```
    p1b,
    data_base2["t_points"], data_base2["y_values"],
    label="y(t)",
    lw=2
)

savefig(p1b, plotsdir(script_name, "single_experiment_nonlinear.png"))
```

5.10 Параметрическое сканирование

Сканируем параметр alpha: для linear alpha -> a для nonlinear alpha -> a2

```
param_grid = Dict(
    :x0 => [32888.0],
    :y0 => [17777.0],

    :tspan => [(0.0, 1.0)],
    :nt => [100],

    :model_type => [:linear, :nonlinear],

    :a => [0.30, 0.55, 0.80, 1.00],     # для linear
    :b => [0.77],
    :c => [0.66],
    :d => [0.44],

    :a2 => [0.15, 0.27, 0.40, 0.60],    # для nonlinear
    :b2 => [0.88],
```

```
    :c2 => [0.68],
    :d2 => [0.37],

    :solver => [Tsit5()],
    :experiment_name => ["parametric_scan"]
)

all_params = dict_list(param_grid)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые model_type: ", param_grid[:model_type])
println("Исследуемые a: ", param_grid[:a])
println("Исследуемые a2: ", param_grid[:a2])
println("="^60)
```

5.11 Запуск всех экспериментов и сбор результатов

```
all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println("Прогресс: $i/$(length(all_params)) | model_type=$(params[:model_type]) |

    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"),
```

```
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    result_summary = merge(
        params,
        Dict(
            :x_final => data["x_final"],
            :y_final => data["y_final"],
            :x_max_abs => data["x_max_abs"],
            :y_max_abs => data["y_max_abs"],
            :norm_final => data["norm_final"],
            :filepath => path
        )
    )
    push!(all_results, result_summary)

    df = DataFrame(
        t = data["t_points"],
        x = data["x_values"],
        y = data["y_values"],
        model_type = fill(string(params[:model_type]), length(data["t_points"])),
        a = fill(params[:a], length(data["t_points"])),
        a2 = fill(params[:a2], length(data["t_points"]))
    )
```

```
    push!(all_dfs, df)
end
```

5.12 Анализ и визуализация результатов сканирования

```
results_df = DataFrame(all_results)
println("\nСводная таблица результатов (первые строки):")
println(first(results_df, 10))
```

Сравнительный график x(t) для всех комбинаций

```
p2 = plot(size=(900, 520), dpi=150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = params[:model_type] == :linear ?
        "linear, a=$(params[:a])" :
        "nonlinear, a2=$(params[:a2])"

    plot!(
```

```
        p2,
        data["t_points"], data["x_values"],
        label=label_text,
        lw=2,
        alpha=0.8
    )
end
plot!(
    p2,
    xlabel="t",
    ylabel="x(t)",
    title="Сканирование: траектории x(t) для разных параметров",
    legend=:outerright,
    grid=true
)
savefig(p2, plotsdir(script_name, "parametric_scan_x_comparison.png"))
```

Сравнительный график y(t) для всех комбинаций

```
p3 = plot(size=(900, 520), dpi=150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )
```

```
    label_text = params[:model_type] == :linear ?
        "linear, a=$(params[:a])" :
        "nonlinear, a2=$(params[:a2])"

    plot!(
        p3,
        data["t_points"], data["y_values"],
        label=label_text,
        lw=2,
        alpha=0.8
    )
end
plot!(
    p3,
    xlabel="t",
    ylabel="y(t)",
    title="Сканирование: траектории y(t) для разных параметров",
    legend=:outerright,
    grid=true
)
savefig(p3, plotsdir(script_name, "parametric_scan_y_comparison.png"))
```

График нормы финального состояния

```
p4 = plot(size=(900, 520), dpi=150)

linear_sub = results_df[results_df.model_type .== :linear, :]
nonlinear_sub = results_df[results_df.model_type .== :nonlinear, :]
```

```
if nrow(linear_sub) > 0
    plot!(
        p4,
        linear_sub.a, linear_sub.norm_final,
        seriestype=:scatter,
        label="linear"
    )
end

if nrow(nonlinear_sub) > 0
    plot!(
        p4,
        nonlinear_sub.a2, nonlinear_sub.norm_final,
        seriestype=:scatter,
        label="nonlinear"
    )
end

plot!(
    p4,
    xlabel="Параметр модели",
    ylabel="norm_final",
    title="Зависимость norm_final от параметра модели",
    legend=:topleft,
    grid=true
)
savefig(p4, plotsdir(script_name, "norm_final_vs_parameter.png"))
```

5.13 Бенчмаркинг

```
println("\n" * "="^60)
println("БЕНЧМАРКИНГ ДЛЯ РАЗНЫХ ПАРАМЕТРОВ")
println("="^60)

benchmark_results = []

for params in all_params
    function benchmark_run()
        u0 = [params[:x0], params[:y0]]

        if params[:model_type] == :linear
            p = (a=params[:a], b=params[:b], c=params[:c], d=params[:d])
            prob = ODEProblem(syst_linear!, u0, params[:tspan], p)
        else
            p = (a=params[:a2], b=params[:b2], c=params[:c2], d=params[:d2])
            prob = ODEProblem(syst_nonlinear!, u0, params[:tspan], p)
        end

        return solve(
            prob,
            params[:solver];
            saveat=LinRange(params[:tspan][1], params[:tspan][2], params[:nt])
        )
    end

    println("\nБенчмарк для model_type=$(params[:model_type]), a=$(params[:a]), a2=$(
```

```
    b = @benchmark $benchmark_run() samples=80 evals=1
    tsec = median(b).time / 1e9
    println(" Медианное время: ", round(tsec; digits=6), " сек")

    push!(benchmark_results, (
        model_type = string(params[:model_type]),
        a = params[:a],
        a2 = params[:a2],
        time = tsec
    ))
end

bench_df = DataFrame(benchmark_results)
```

График времени вычисления

```
p5 = plot(size=(900, 520), dpi=150)

bench_linear = bench_df[bench_df.model_type .== "linear", :]
bench_nonlinear = bench_df[bench_df.model_type .== "nonlinear", :]

if nrow(bench_linear) > 0
    plot!(
        p5,
        bench_linear.a, bench_linear.time,
        seriestype=:scatter,
        label="linear"
    )
end
```

```
if nrow(bench_nonlinear) > 0
    plot!(
        p5,
        bench_nonlinear.a2, bench_nonlinear.time,
        seriestype=:scatter,
        label="nonlinear"
    )
end

plot!(
    p5,
    xlabel="Параметр модели",
    ylabel="Время вычисления, сек",
    title="Зависимость времени решения ODE от параметров модели",
    legend=:topleft,
    grid=true
)
savefig(p5, plotsdir(script_name, "computation_time_vs_parameter.png"))
```

5.14 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") base_params base_params2 param_grid al
@save datadir(script_name, "all_plots.jld2") p1 p1b p2 p3 p4 p5

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)
```

```
println("\nРезультаты сохранены в:")
println(" • data/$(script_name)/single/ - базовые эксперименты")
println(" • data/$(script_name)/parametric_scan/ - параметрическое сканирование")
println(" • data/$(script_name)/all_results.jld2 - сводные данные")
println(" • plots/$(script_name)/ - все графики")
println(" • data/$(script_name)/all_plots.jld2 - объекты графиков")
println("\nДля анализа результатов используйте:")
println(" using JLD2, DataFrames")
println(" @load \"data/$(script_name)/all_results.jld2\"")
println(" println(results_df)")
```

5.15 Базовые эксперименты

5.15.1 Линейная модель (model_type = linear)

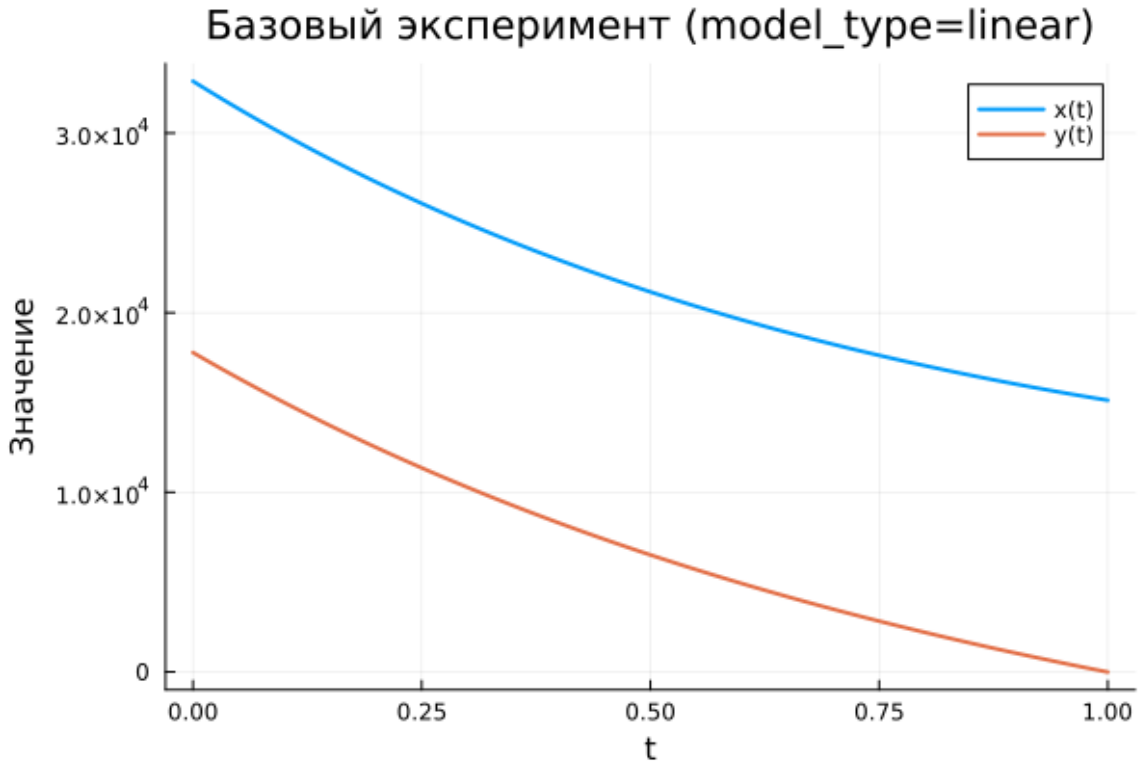

Рисунок 5.1: Базовый эксперимент (linear)

На графике представлена эволюция переменных $x(t)$ и $y(t)$ в рамках линейной модели. Обе функции со временем уменьшаются.

Функция $x(t)$ убывает плавно и остаётся положительной на протяжении всего интервала интегрирования. Это свидетельствует о постепенном ослаблении системы без резких скачков.

Функция $y(t)$ уменьшается более быстро и к концу интервала становится близкой к нулю. Подобное поведение характерно для линейных динамических систем, где решения экспоненциально стремятся к устойчивому состоянию.

Таким образом, линейная модель демонстрирует устойчивую и предсказуемую динамику.

5.15.2 Нелинейная модель (model_type = nonlinear)

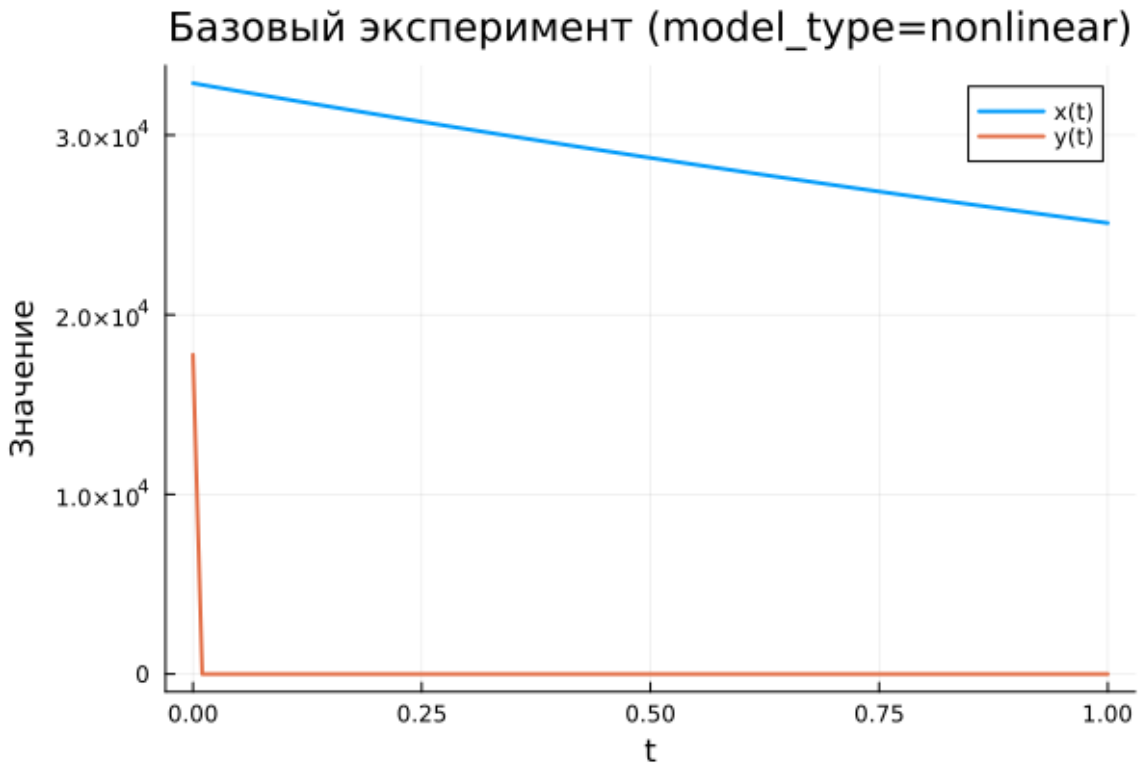

Рисунок 5.2: Базовый эксперимент (nonlinear)

Для нелинейной модели характерна иная картина. Переменная $x(t)$ уменьшается значительно медленнее и остаётся достаточно большой на всём временном интервале.

Переменная $y(t)$ практически сразу стремится к нулю. После начального этапа её значение остаётся близким к нулю.

Такой эффект объясняется присутствием нелинейных членов, которые усиливают скорость подавления одной из переменных. В результате динамика системы определяется главным образом функцией $x(t)$.

5.16 Параметрическое сканирование

5.16.1 Траектории $x(t)$ для различных параметров

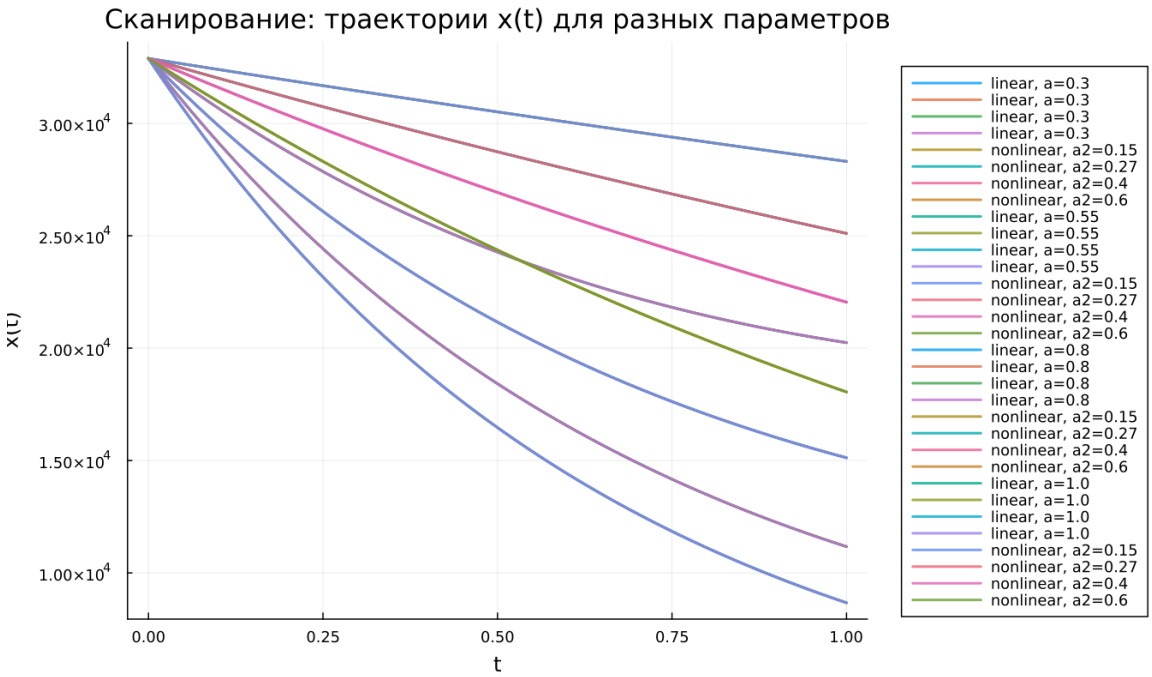

Рисунок 5.3: Сканирование траекторий x(t)

Было выполнено исследование влияния параметров модели на траектории решения.

В линейной модели изменялся параметр a, а в нелинейной — параметр a_2.

Наблюдается следующая закономерность: при увеличении параметра скорость убывания функции $x(t)$ возрастает, что приводит к более быстрому снижению значения переменной.

Основные наблюдения:

- при небольших значениях параметра уменьшение $x(t)$ происходит медленно;
- при увеличении параметра траектории становятся более крутыми;

- различия между кривыми становятся заметными уже на середине временного интервала.

В нелинейной системе влияние параметров сохраняется, однако форма траекторий становится более сложной.

5.16.2 Траектории $y(t)$ для различных параметров

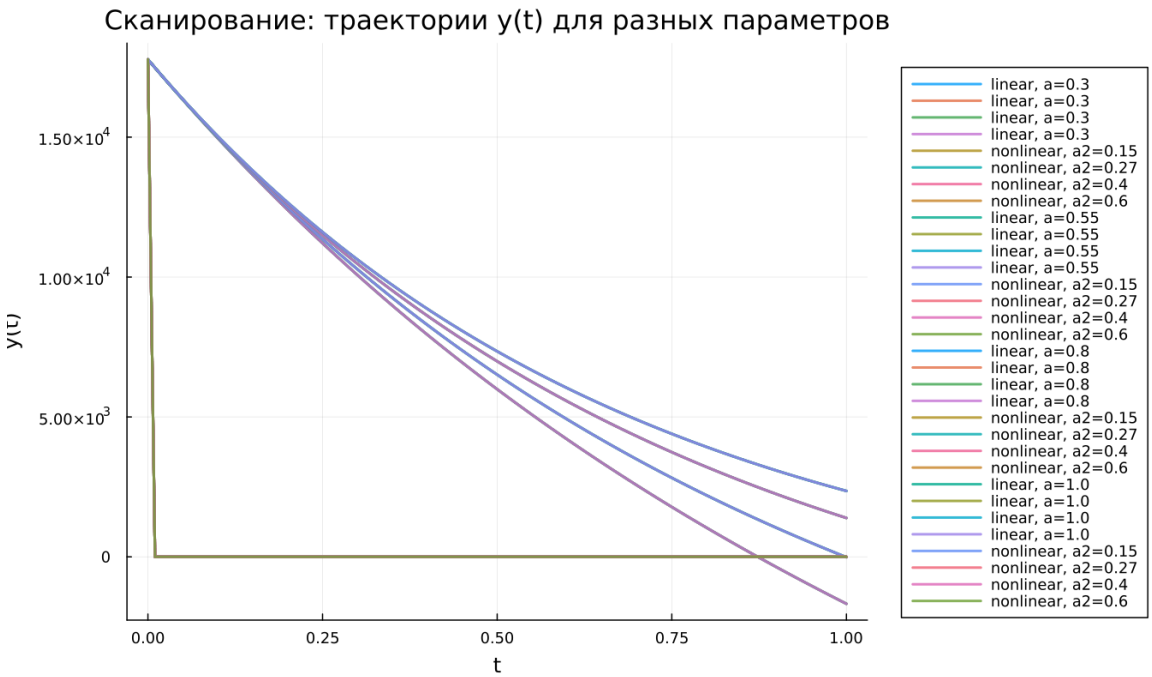

Рисунок 5.4: Сканирование траекторий y(t)

Аналогичное исследование было выполнено для переменной $y(t)$.

В линейной модели изменение параметров приводит к постепенному изменению формы кривых: при увеличении параметра функция быстрее стремится к нулю.

В нелинейной модели поведение отличается: значение $y(t)$ практически сразу становится близким к нулю вне зависимости от параметра.

Это свидетельствует о доминирующем влиянии нелинейных членов системы.

5.17 Время вычислений

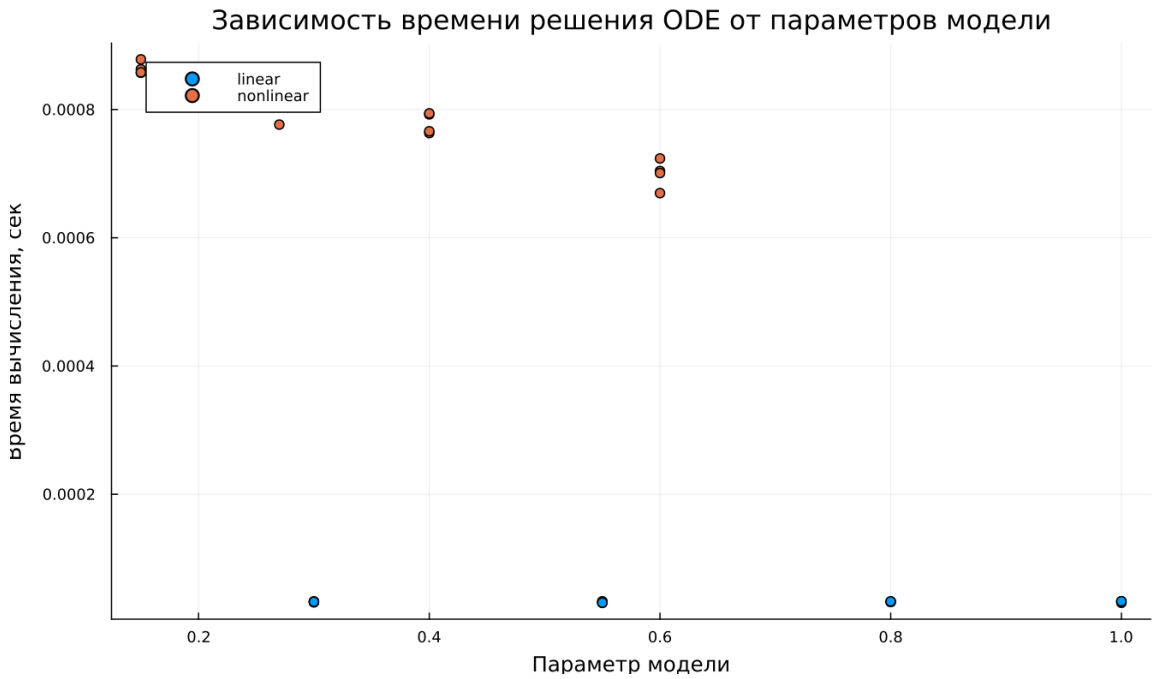

Рисунок 5.5: Зависимость времени вычисления

Был проведён анализ времени численного решения системы дифференциальных уравнений при различных параметрах.

Результаты показывают:

- линейная модель вычисляется значительно быстрее;
- время решения линейной системы составляет порядка 10^{-5} секунд;
- для нелинейной системы требуется около $7 \cdot 10^{-4}$–$9 \cdot 10^{-4}$ секунд.

Несмотря на различия, абсолютное время вычислений остаётся крайне малым.

5.18 Анализ итоговой метрики norm_final

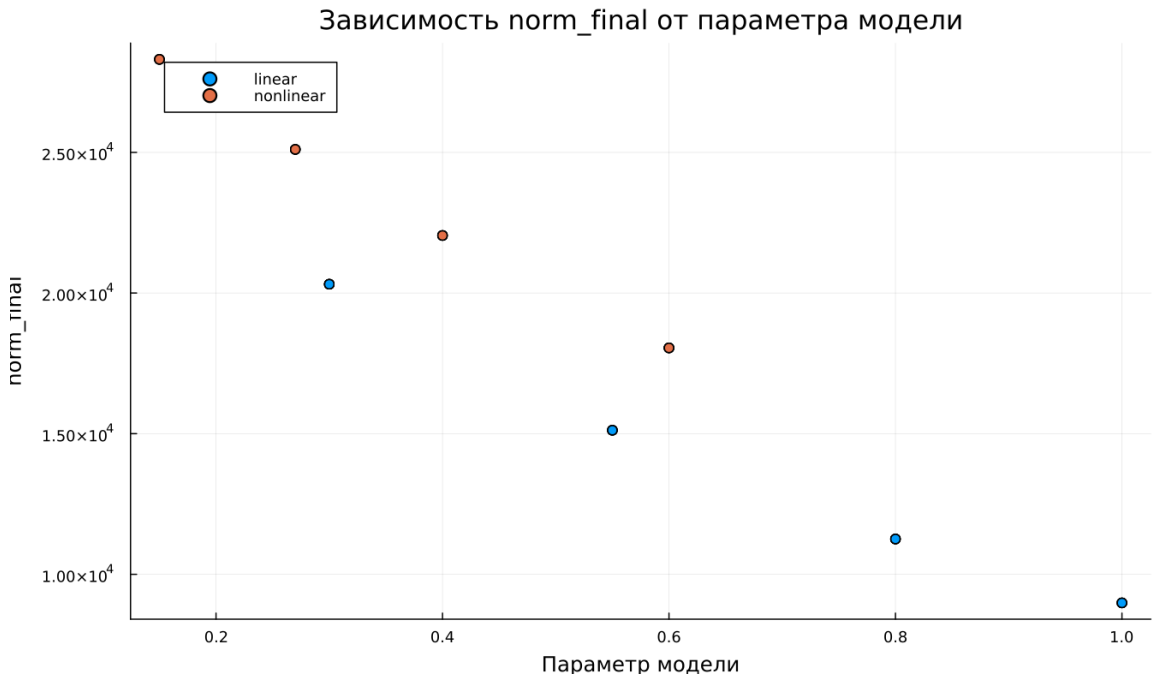

Рисунок 5.6: Зависимость norm_final

В качестве итоговой характеристики системы использовалась метрика

$$\text{norm_final} = \sqrt{x(t_{final})^2 + y(t_{final})^2}$$

Она отражает величину состояния системы в конце временного интервала. Анализ показывает:

- при увеличении параметра значение метрики уменьшается;
- линейная модель демонстрирует более быстрое снижение нормы;
- в нелинейной модели значения метрики остаются выше.

Это означает, что линейная система быстрее достигает состояния покоя.

6 Выводы

1. Линейная модель характеризуется плавным и предсказуемым уменьшением переменных $x(t)$ и $y(t)$. При этом функция $y(t)$ убывает быстрее.

2. Нелинейная модель демонстрирует качественно иную динамику: переменная $y(t)$ практически мгновенно стремится к нулю, после чего система определяется главным образом функцией $x(t)$.

3. Параметры системы существенно влияют на скорость затухания решений. Их увеличение приводит к более быстрому снижению значений переменных.

4. В нелинейной модели влияние параметров на переменную $y(t)$ оказывается слабым из-за доминирования нелинейных членов.

5. Численные эксперименты показывают, что линейная система решается быстрее, однако даже для нелинейной модели время вычислений остаётся очень малым.

6. Итоговая метрика norm_final уменьшается при увеличении параметров, что свидетельствует об усилении затухания динамики системы.

Полученные результаты согласуются с теоретическими ожиданиями для линейных и нелинейных дифференциальных моделей.

Список литературы

1. Законы Осипова — Ланчестера
2. Дифференциальные уравнения динамики боя
3. Элементарные модели боя